

1. Set the **Peer Session Identifier** in the **Session Context** to the value **PBKDFParamResponse.responderSessionId**.
2. Generate the **Crypto_PAKEValues_Initiator** according to the **PBKDFParamResponse.pbkdf_parameters**
3. Using **Crypto_PAKEValues_Initiator**, generate **pA := Crypto_pA(Crypto_PAKEValues_Initiator)**
4. Send a message with the appropriate **Protocol Id** and **Protocol Opcode** from Table 18, “Secure Channel Protocol Opcodes” whose payload is the TLV-encoded **pake-1-struct Pake1** with an anonymous tag for the outermost struct.

```
Pake1 =
{
  pA (1) = pA,
}
```

Pake2

```
pake-2-struct => STRUCTURE [ tag-order ]
{
  pB [1] : OCTET STRING [ length CRYPTO_PUBLIC_KEY_SIZE_BYTES ],
  cB [2] : OCTET STRING [ length CRYPTO_HASH_LEN_BYTES],
}
```

On receipt of **Pake1**, the responder SHALL:

1. Compute **pB := Crypto_pB(Crypto_PAKEValues_Responder)** using the passcode verifier indicated in **PBKDFParamRequest**
2. Compute **TT := Crypto_Transcript(PBKDFParamRequest, PBKDFParamResponse, Pake1.pA, pB)**
3. Compute **(cA, cB, Ke) := Crypto_P2(TT, Pake1.pA, pB)**
4. Send a message with the appropriate **Protocol Id** and **Protocol Opcode** from Table 18, “Secure Channel Protocol Opcodes” whose payload is the TLV-encoded **pake-2-struct Pake2** with an anonymous tag for the outermost struct.

```
Pake2 =
{
  pB (1) = pB,
  cB (2) = cB,
}
```

Pake3

```
pake-3-struct => STRUCTURE [ tag-order ]
{
```